In [1]:
```
class Sorting:
    def bubble(self,arr):
        l=len(arr)
        for i in range(l):
            flag=1
            for j in range(l-i-1):
                if arr[j+1]<arr[j]:
                    arr[j+1],arr[j]=arr[j],arr[j+1]
                    flag=0
            if flag==1:
                return arr
        return arr
obj=Sorting()
obj.bubble([5,2,7,9,1,0,3,4,8,6,6,3])
```

Out[1]: [0, 1, 2, 3, 3, 4, 5, 6, 6, 7, 8, 9]

In [3]:
```
class Sorting:
    def insertion(self,arr):
        l=len(arr)
        for i in range(1,l):
            val=arr[i]
            gap=i
            while gap>0 and arr[gap-1]>val:
                arr[gap]=arr[gap-1]
                gap-=1
            arr[gap]=val
        return arr
obj=Sorting()
obj.insertion([5,2,7,9,1,0,3,4,8,6,6,3])
```

Out[3]: [0, 1, 2, 3, 3, 4, 5, 6, 6, 7, 8, 9]

In [6]:
```
class Sorting:
    def selection(self,arr):
        l=len(arr)
        for i in range(l):
            ind=i
            for j in range(i+1,l):
                if arr[j]<arr[ind]:
                    ind=j
            arr[ind],arr[i]=arr[i],arr[ind]
        return arr
obj=Sorting()
obj.selection([5,2,7,9,1,0,3,4,8,6,6,3])
```

Out[6]: [0, 1, 2, 3, 3, 4, 5, 6, 6, 7, 8, 9]

In [3]:
```
class Sorting:
    def quick(self,arr):
        l=len(arr)
        if l<=1:
            return arr
        else:
            pivot=arr[0]
            left=[i for i in arr[1:] if i<pivot]
            right=[i for i in arr[1:] if i>=pivot]
            return self.quick(left)+[pivot]+self.quick(right)
obj=Sorting()
obj.quick([5,2,7,9,1,0,3,4,8,6,6,3])
```

Out[3]: [0, 1, 2, 3, 3, 4, 5, 6, 6, 7, 8, 9]

In [1]:
```
class Solution:
    def merge(self,left,right):
        i,j=0,0
        result=[]
        while(i<len(left) and j<len(right)):
            if left[i]<=right[j]:
                result+=[left[i]]
                i+=1
            else:
                result+=[right[j]]
                j+=1
        result+=left[i:]+right[j:]
        return result

    def mergesort(self,arr):
        if len(arr)<=1:
            return arr
        mid=len(arr)//2
        left=self.mergesort(arr[:mid])
        right=self.mergesort(arr[mid:])
        return self.merge(left,right)
obj=Solution()
arr=[1,2,5,-1,-4,0,8,5]
obj.mergesort(arr)
```

Out[1]: [-4, -1, 0, 1, 2, 5, 5, 8]

In [1]:
```
class Solution:
    def quicksort(self,arr,left,right):
        if left<right:
            pos=self.partition(arr,left,right)
            self.quicksort(arr,left,pos-1)
            self.quicksort(arr,pos+1,right)
        return arr
    def partition(self,arr,left,right):
        i=left
        j=right-1
        pivot=arr[right]
        while i<j:
            while i<right and arr[i]<pivot:
                i+=1
            while arr[j]>=pivot and j>=i:
                j-=1
            if i<j:
                arr[i],arr[j]=arr[j],arr[i]
        if arr[i]>pivot:
            arr[i],arr[right]=arr[right],arr[i]
        return i
obj=Solution()
arr=[22,11,88,66,55,77,33,44,44,44,4]
obj.quicksort(arr,0,len(arr)-1)
```

Out[1]: [4, 11, 22, 33, 44, 44, 44, 55, 66, 77, 88]

In [5]:
```
class Solution:
    def quick(self,arr):
        if len(arr)<=1:
            return arr
        else:
            pivot=arr[0]
            left=[i for i in arr[1:] if i<=pivot]
            right=[i for i in arr[1:] if i>pivot]
            return self.quick(left)+[pivot]+self.quick(right)
obj=Solution()
obj.quick([1,2,5,-1,-4,0,8,5])
```

Out[5]: [-4, -1, 0, 1, 2, 5, 5, 8]

In [11]:
```
class Solution:
    def quicksort(self,arr,left,right):
        if left<right:
            pos=self.partition(arr,left,right)
            self.quicksort(arr,left,pos-1)
            self.quicksort(arr,pos+1,right)
        return arr
    def partition(self,arr,left,right):
        i=left
        j=right-1
        pivot=arr[right]
        while i<j:
            while i<right and arr[i]<pivot:
                i+=1
            while j>=i and arr[j]>pivot:
                j-=1
            if i<=j:
                arr[i],arr[j]=arr[j],arr[i]
        if arr[i]>pivot:
            arr[i],arr[right]=arr[right],arr[i]
        return i
obj=Solution()
arr=[1,2,5,-1,-4,0,8,5]
obj.quicksort(arr,0,len(arr)-1)
```

Out[11]: [-4, -1, 0, 1, 2, 5, 5, 8]

In [13]:
```
class Solution:
    def divide(self,arr):
        if len(arr)<=1:
            return arr
        mid=len(arr)//2
        left=self.divide(arr[:mid])
        right=self.divide(arr[mid:])
        return self.conquer(left,right)

    def conquer(self,left,right):
        res=[]
        i,j=0,0
        while(i<len(left) and j<len(right)):
            if left[i]<right[j]:
                res+=[left[i]]
                i+=1
            else:
                res+=[right[j]]
                j+=1
        res+=left[i:]+right[j:]
        return res
obj=Solution()
obj.divide([1,2,5,-1,-4,0,8,5])
```

Out[13]: [-4, -1, 0, 1, 2, 5, 5, 8]

In [1]:
```
from collections import Counter
def TreeConstructor(strArr):
    parent = []
    child = []

    for i in strArr:
        child.append(int(i[1]))
        parent.append(int(i[3]))
    for k,v in Counter(parent).items():
        if v > 2:
            return False
    for k,v in Counter(child).items():
        if v > 1:
            return False
    return True

print(TreeConstructor(["(1,2)", "(2,4)", "(5,7)", "(7,2)", "(9,5)"]))
print(TreeConstructor(["(1,2)", "(3,2)", "(2,12)", "(5,2)"]))
```

```
True
False
```

In [2]:
```
def Calculator(strParam):
    return int(eval(strParam))
print(Calculator("6*(4/2)+3*1"))
```

```
15
```

In [8]:
```
class Sorting:
    def bubbleSort(self,arr):
        print(["unsorted array : ",arr])
        l=len(arr)
        for i in range(l):
            flag=0
            for j in range(l-i-1):
                if arr[j+1]<arr[j]:
                    arr[j],arr[j+1]=arr[j+1],arr[j]
                    flag=1
            if flag==0:
                return ["sorted array : ",arr]
        return ["sorted array : ",arr]
arr=[-5,2,5,1,7,9,0,-1,-1,0]
obj=Sorting()
obj.bubbleSort(arr)
```

```
['unsorted array : ', [-5, 2, 5, 1, 7, 9, 0, -1, -1, 0]]
```

Out[8]:
```
['sorted array : ', [-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]]
```

In [11]:
```
class Sorting:
    def insertionSort(self,arr):
        print(["unsorted array : ",arr])
        l=len(arr)
        for i in range(1,l):
            gap=i
            value=arr[i]
            while gap>0 and arr[gap-1]>value:
                arr[gap]=arr[gap-1]
                gap-=1
            arr[gap]=value
        return ["sorted array : ",arr]
arr=[-5,2,5,1,7,9,0,-1,-1,0]
obj=Sorting()
obj.insertionSort(arr)
```

```
['unsorted array : ', [-5, 2, 5, 1, 7, 9, 0, -1, -1, 0]]
```

Out[11]:
```
['sorted array : ', [-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]]
```

In [17]:
```
class Sorting:
    def selection(self,arr):
        print(["unsorted array : ",arr])
        l=len(arr)
        for i in range(l-1):
            min_index=i
            for j in range(i+1,l):
                if arr[j]<arr[min_index]:
                    min_index=j
            arr[min_index],arr[i]=arr[i],arr[min_index]
        return ["sorted array : ",arr]
arr=[-5,2,5,1,7,9,0,-1,-1,0]
obj=Sorting()
obj.selection(arr)
```

```
['unsorted array : ', [-5, 2, 5, 1, 7, 9, 0, -1, -1, 0]]
```

Out[17]:
```
['sorted array : ', [-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]]
```

In [31]:
```
class sorting:
    print(["unsorted array : ",arr])
    def divide(self,arr):
        if len(arr)<=1:
            return arr
        mid=len(arr)//2
        left=self.divide(arr[:mid])
        right=self.divide(arr[mid:])
        return self.conquer(left,right)
    def conquer(self,left,right):
        i,j=0,0
        res=[]
        while i<len(left) and j<len(right):
            if left[i]<right[j]:
                res+=[left[i]]
                i+=1
            else:
                res+=[right[j]]
                j+=1
        res+=left[i:]+right[j:]
        return res
arr=[-5,2,5,1,7,9,0,-1,-1,0]
obj=sorting()
obj.divide(arr)
```

```
['unsorted array : ', [-5, 2, 5, 1, 7, 9, 0, -1, -1, 0]]
```

Out[31]:
```
[-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]
```

In [39]:
```
class Sorting:
    def quick(self,arr):
        self.a=5
        if len(arr)<=1:
            return arr
        pivot=arr[0]
        left=[i for i in arr[1:] if i<pivot]
        right=[i for i in arr[1:] if i>=pivot]
        return self.quick(left)+[pivot]+self.quick(right)
obj=Sorting()
arr=[-5,2,5,1,7,9,0,-1,-1,0]
obj.quick(arr)
```

Out[39]:
```
[-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]
```

In [1]:
```
class Sorting:
    def quick(self,arr,left,right):
        if left<right:
            pos=self.partition(arr,left,right)
            self.quick(arr,left,pos-1)
            self.quick(arr,pos+1,right)
        return arr
    def partition(self,arr,left,right):
        i=left
        j=right-1
        pivot=arr[right]
        while i<j:
            while i<right and arr[i]<pivot:
                i+=1
            while j>=i and arr[j]>=pivot:
                j-=1
            if i<=j:
                arr[i],arr[j]=arr[j],arr[i]
        if arr[i]>pivot:
            arr[i],arr[right]=arr[right],arr[i]
        return i
obj=Sorting()
arr=[-5,2,5,1,7,9,0,-1,-1,0]
obj.quick(arr,0,len(arr)-1)
```

Out[1]:
```
[-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]
```

In [2]:

```
class Sorting:
    def buildHeap(self,arr,n):
        for i in range(n//2-1,-1,-1):
            self.heapify(arr,n,i)
        print("maxheap:",arr)
    def heapify(self,arr,n,i):
        left=2*i+1
        right=2*i+2
        largest=i
        if left<n and arr[largest]<arr[left]:
            largest=left
        if right<n and arr[largest]<arr[right]:
            largest=right
        if largest!=i:
            arr[largest],arr[i]=arr[i],arr[largest]
            self.heapify(arr,n,largest)
    def heapSort(self,arr,n):
        self.buildHeap(arr,n)
        for i in range(n-1,-1,-1):
            arr[i],arr[0]=arr[0],arr[i]
            self.heapify(arr,i,0)
        return "sorted array is :  ",arr
arr=[4,2,7,45,-1,-34,0,0,23]
n=len(arr)
obj=Sorting()
obj.heapSort(arr,n)
```

```
maxheap: [45, 23, 7, 4, -1, -34, 0, 0, 2]
```

Out[2]:

```
('sorted array is :  ', [-34, -1, 0, 0, 2, 4, 7, 23, 45])
```

In [1]:

```
class Sorting:
    def bubble(self,arr):
        l=len(arr)
        for i in range(l):
            flag=1
            for j in range(l-i-1):
                if arr[j]>arr[j+1]:
                    arr[j],arr[j+1]=arr[j+1],arr[j]
                    flag=0
            if flag==1:
                return arr
arr=[3,-1,2,-2,2,0,9,18,10,-45]
obj=Sorting()
obj.bubble(arr)
```

Out[1]:

```
[-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]
```

In [8]:

```
class Sorting:
    def insertion(self,arr):
        l=len(arr)
        for i in range(1,l):
            value=arr[i]
            gap=i
            while gap>0 and arr[gap-1]>value:
                arr[gap]=arr[gap-1]
                gap-=1
            arr[gap]=value
        return arr
arr=[3,-1,2,-2,2,0,9,18,10,-45]
obj=Sorting()
obj.insertion(arr)
```

Out[8]:

```
[-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]
```

In [13]:

```
class Sorting:
    def selection(self,arr):
        l=len(arr)
        for i in range(l-1):
            ind=i
            for j in range(i+1,l):
                if arr[j]<arr[ind]:
                    ind=j
            arr[ind],arr[i]=arr[i],arr[ind]
        return arr
arr=[3,-1,2,-2,2,0,9,18,10,-45]
obj=Sorting()
obj.selection(arr)
```

Out[13]:

```
[-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]
```

In [15]:
```
class Sorting:
    def quick(self,arr):
        if len(arr)<=1:
            return arr
        pivot=arr[0]
        left=[i for i in arr[1:] if i<pivot]
        right=[i for i in arr[1:] if i>=pivot]
        return self.quick(left)+[pivot]+self.quick(right)
arr=[3,-1,2,-2,2,0,9,18,10,-45]
obj=Sorting()
obj.quick(arr)
```

Out[15]: [-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]

In [17]:
```
class Sorting:
    def divide(self,arr):
        if len(arr)<=1:
            return arr
        mid=len(arr)//2
        left=self.divide(arr[:mid])
        right=self.divide(arr[mid:])
        return self.merge(left,right)
    def merge(self,left,right):
        i,j=0,0
        res=[]
        while i<len(left) and j<len(right):
            if left[i]<right[j]:
                res+=[left[i]]
                i+=1
            else:
                res+=[right[j]]
                j+=1
        res+=left[i:]+right[j:]
        return res
arr=[3,-1,2,-2,2,0,9,18,10,-45]
obj=Sorting()
obj.divide(arr)
```

Out[17]: [-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]

In [1]:
```
class Sorting:
    def quick(self,arr,left,right):
        if left<right:
            pos=self.partition(arr,left,right)
            self.quick(arr,left,pos-1)
            self.quick(arr,pos+1,right)
        return arr
    def partition(self,arr,left,right):
        i=left
        j=right-1
        pivot=arr[right]
        while i<j:
            while i<right and arr[i]<pivot:
                i+=1
            while j>=i and arr[j]>pivot:
                j-=1
            if i<j:
                arr[i],arr[j]=arr[j],arr[i]
        if arr[i]>pivot:
            arr[i],arr[right]=arr[right],arr[i]
        return i
arr=[3,-1,2,-2,2,0,9,18,10,-45]
obj=Sorting()
obj.quick(arr,0,len(arr)-1)
```

Out[1]: [-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]

In [5]:
```
class Sorting:
    def buildheap(self,arr,n):
        for i in range(n//2-1,-1,-1):
            self.heapify(arr,n,i)
    def heapify(self,arr,n,i):
        largest=i
        left=2*i+1
        right=2*i+2
        if left<n and arr[left]>arr[largest]:
            largest=left
        if right<n and arr[right]>arr[largest]:
            largest=right
        if largest!=i:
            arr[largest],arr[i]=arr[i],arr[largest]
            self.heapify(arr,n,largest)
    def heapsort(self,arr,n):
        self.buildheap(arr,n)
        for i in range(n-1,-1,-1):
            arr[0],arr[i]=arr[i],arr[0]
            self.heapify(arr,i,0)
        return arr
arr=[3,-1,2,-2,2,0,9,18,10,-45]
obj=Sorting()
obj.heapsort(arr,len(arr)-1)
```

Out[5]: [-2, -1, 0, 2, 2, 3, 9, 10, 18, -45]

In [4]:
```
class Sorting:
    def divide(self,arr):
        if len(arr)<=1:
            return arr
        mid=len(arr)//2
        left=self.divide(arr[:mid])
        right=self.divide(arr[mid:])
        return self.merge(left,right)
    def merge(self,left,right):
        i,j=0,0
        res=[]
        while i<len(left) and j<len(right):
            if left[i]<right[j]:
                res+=[left[i]]
                i+=1
            else:
                res+=[right[j]]
                j+=1
        res+=left[i:]+right[j:]
        return res
obj=Sorting()
arr=[9,3,1,8,4,6,0,-1,-4,-2,-1,-1]
obj.divide(arr)
```

Out[4]: [-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]

In [1]:
```
class Sorting:
    def quick(self,arr):
        if len(arr)<=1:
            return arr
        pivot=arr[0]
        left=[i for i in arr[1:] if i<pivot]
        right=[i for i in arr[1:] if i>=pivot]
        return self.quick(left)+[pivot]+self.quick(right)
obj=Sorting()
arr=[9,3,1,8,4,6,0,-1,-4,-2,-1,-1]
obj.quick(arr)
```

Out[1]: [-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]

In [9]:
```
class Sorting:
    def quick(self,arr,left,right):
        if left<right:
            pos=self.partition(arr,left,right)
            self.quick(arr,left,pos-1)
            self.quick(arr,pos+1,right)
        return arr
    def partition(self,arr,left,right):
        i=left
        j=right-1
        pivot=arr[right]
        while i<j:
            while i<right and arr[i]<pivot:
                i+=1
            while j>=i and arr[j]>=pivot:
                j-=1
            if i<j:
                arr[i],arr[j]=arr[j],arr[i]
        if arr[i]>pivot:
            arr[i],arr[right]=arr[right],arr[i]
        return i
obj=Sorting()
arr=[9,3,1,8,4,6,0,-1,-4,-2,-1,-1]
obj.quick(arr,0,len(arr)-1)
```

Out[9]: [-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]

In [10]:
```
class Sorting:
    def bubble(self,arr):
        l=len(arr)
        for i in range(l):
            flag=1
            for j in range(l-i-1):
                if arr[j]>arr[j+1]:
                    arr[j],arr[j+1]=arr[j+1],arr[j]
                    flag=0
            if flag==1:
                return arr
        return arr
obj=Sorting()
arr=[9,3,1,8,4,6,0,-1,-4,-2,-1,-1]
obj.bubble(arr)
```

Out[10]: [-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]

In [11]:
```
class Sorting:
    def insertion(self,arr):
        l=len(arr)
        for i in range(1,l):
            gap=i
            val=arr[gap]
            while gap>0 and arr[gap-1]>val:
                arr[gap]=arr[gap-1]
                gap-=1
            arr[gap]=val
        return arr
obj=Sorting()
arr=[9,3,1,8,4,6,0,-1,-4,-2,-1,-1]
obj.insertion(arr)
```

Out[11]:
```
[-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]
```

In [18]:
```
class Sorting:
    def selection(self,arr):
        l=len(arr)
        for i in range(l):
            index=i
            for j in range(i+1,l):
                if arr[j]<arr[index]:
                    index=j
            arr[i],arr[index]=arr[index],arr[i]
        return arr
obj=Sorting()
arr=[9,3,1,8,4,6,0,-1,-4,-2,-1,-1]
obj.selection(arr)
```

Out[18]:
```
[-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]
```

In [2]:
```
class Solution:
    def quick(self,arr):
        if len(arr)<=1:
            return arr
        pivot=arr[0]
        left=[i for i in arr[1:] if i<pivot]
        right=[i for i in arr[1:] if i>=pivot]
        return self.quick(left)+[pivot]+self.quick(right)
obj=Solution()
arr=[5,43,7,1,77,0,-8,34,-1213123]
obj.quick(arr)
```

Out[2]:
```
[-1213123, -8, 0, 1, 5, 7, 34, 43, 77]
```

In [3]:
```
class Solution:
    def quicksort(self,arr,left,right):
        if left<right:
            pos=self.partition(arr,left,right)
            self.quicksort(arr,left,pos-1)
            self.quicksort(arr,pos+1,right)
        return arr
    def partition(self,arr,left,right):
        pivot=arr[right]
        i=left
        j=right-1
        while i<j:
            while i<right and arr[i]<pivot:
                i+=1
            while j>i and arr[j]>=pivot:
                j-=1
            if i<j:
                arr[i],arr[j]=arr[j],arr[i]
        if arr[i]>pivot:
            arr[i],arr[right]=arr[right],arr[i]
        return i
obj=Solution()
arr=[2,4,64,12,231,-89,0,654]
obj.quicksort(arr,0,len(arr)-1)
```

Out[3]:
```
[-89, 0, 2, 4, 12, 64, 231, 654]
```

In [4]:
```
class Solution:
    def divide(self,arr):
        if len(arr)<=1:
            return arr
        mid=len(arr)//2
        left=self.divide(arr[:mid])
        right=self.divide(arr[mid:])
        return self.conquer(left,right)
    def conquer(self,left,right):
        i,j=0,0
        res=[]
        while i<len(left) and j<len(right):
            if left[i]<right[j]:
                res+=[left[i]]
                i+=1
            else:
                res+=[right[j]]
                j+=1
        res+=left[i:]+right[j:]
        return res
obj=Solution()
arr=[1,23,5345121,12,213,-98,0,456,34]
obj.divide(arr)
```

Out[4]:
```
[-98, 0, 1, 12, 23, 34, 213, 456, 5345121]
```

In [20]:
```
class Solution:
    def buildheap(self,arr,n):
        for i in range(n//2-1,-1,-1):
            self.heapify(arr,n,i)
    def heapify(self,arr,n,i):
        largest=i
        left=2*i+1
        right=2*i+2
        if left<n and arr[left]>arr[largest]:
            largest=left
        if right<n and arr[right]>arr[largest]:
            largest=right
        if largest!=i:
            arr[largest],arr[i]=arr[i],arr[largest]
            self.heapify(arr,n,largest)
    def heapsort(self,arr,n):
        self.buildheap(arr,n)
        for i in range(n-1,-1,-1):
            arr[0],arr[i]=arr[i],arr[0]
            self.heapify(arr,i,0)
        return arr
obj=Solution()
arr=[1,23,5345121,12,213,-98,0,456,34,45,4567,123125,1]
obj.heapsort(arr,len(arr))
```

Out[20]:
```
[-98, 0, 1, 1, 12, 23, 34, 45, 213, 456, 4567, 123125, 5345121]
```

In [3]:
```
class Sorting:
    def buildheap(self,arr,n):
        for i in range(n//2-1,-1,-1):
            self.heapify(arr,n,i)
    def heapify(self,arr,n,i):
        largest=i
        left=2*i+1
        right=2*i+2
        if left<n and arr[left]>arr[largest]:
            largest=left
        if right<n and arr[right]>arr[largest]:
            largest=right
        if largest!=i:
            arr[largest],arr[i]=arr[i],arr[largest]
            self.heapify(arr,n,largest)
    def heapsort(self,arr,n):
        self.buildheap(arr,n)
        for i in range(n-1,-1,-1):
            arr[0],arr[i]=arr[i],arr[0]
            self.heapify(arr,i,0)
        return arr
arr=[1,23,534,12,213,-98,0,456,34]
obj=Sorting()
obj.heapsort(arr,len(arr))
```

Out[3]:
```
[-98, 0, 1, 12, 23, 34, 213, 456, 534]
```

In [2]:
```
class Solution:
    def buildTree(self,arr,n):
        for i in range(n//2-1,-1,-1):
            self.heapify(arr,n,i)
        print(arr)
    def heapify(self,arr,n,i):
        largest=i
        left=2*i+1
        right=2*i+2
        if left<n and arr[left]>arr[largest]:
            largest=left
        if right<n and arr[right]>arr[largest]:
            largest=right
        if largest!=i:
            arr[largest],arr[i]=arr[i],arr[largest]
            self.heapify(arr,n,largest)
    def heapsort(self,arr,n):
        self.buildTree(arr,n)
        for i in range(n-1,-1,-1):
            arr[i],arr[0]=arr[0],arr[i]
            self.heapify(arr,i,0)
        return arr
obj=Solution()
arr=[1,23,534,12,213,-98,0,456,34]
obj.heapsort(arr,len(arr))
```

```
[534, 456, 1, 34, 213, -98, 0, 12, 23]
```

Out[2]:
```
[-98, 0, 1, 12, 23, 34, 213, 456, 534]
```

In [9]:
```
class Solution:
    def divide(self,arr):
        if len(arr)<=1:
            return arr
        mid=len(arr)//2
        left=self.divide(arr[:mid])
        right=self.divide(arr[mid:])
        return self.conquer(left,right)
    def conquer(self,left,right):
        i,j=0,0
        res=[]
        while i<len(left) and j<len(right):
            if left[i]<right[j]:
                res+=[left[i]]
                i+=1
            else:
                res+=[right[j]]
                j+=1
        res+=left[i:]+right[j:]
        return res
obj=Solution()
arr=[1,23,534,12,213,-98,0,456,34]
obj.divide(arr)
```

Out[9]:
```
[-98, 0, 1, 12, 23, 34, 213, 456, 534]
```

In [10]:
```
class Solution:
    def quick(self,arr):
        if len(arr)<=1:
            return arr
        pivot=arr[0]
        left=[i for i in arr[1:] if i<pivot]
        right=[i for i in arr[1:] if i>=pivot]
        return self.quick(left)+[pivot]+self.quick(right)
obj=Solution()
arr=[1,23,534,12,213,-98,0,456,34]
obj.quick(arr)
```

Out[10]:
```
[-98, 0, 1, 12, 23, 34, 213, 456, 534]
```

In [12]:
```
class Solution:
    def quick(self,arr,left,right):
        if left<right:
            pos=self.partition(arr,left,right)
            self.quick(arr,left,pos-1)
            self.quick(arr,pos+1,right)
        return arr
    def partition(self,arr,left,right):
        i=left
        j=right-1
        pivot=arr[right]
        while i<j:
            while i<j and arr[i]<pivot:
                i+=1
            while j>=i and arr[j]>pivot:
                j-=1
            if i<j:
                arr[i],arr[j]=arr[j],arr[i]
        if arr[i]>pivot:
            arr[i],arr[right]=arr[right],arr[i]
        return i
obj=Solution()
arr=[1,23,534,12,213,-98,0,456,34]
obj.quick(arr,0,len(arr)-1)
```

Out[12]:
```
[-98, 0, 1, 12, 23, 34, 213, 456, 534]
```

In [33]:
```
class ListNode:
    def __init__(self,val):
        self.val=val
        self.next=None
class LinkedList:
    def __init__(self):
        self.head=None
    def printList(self):
        itr=self.head
        if not itr: return 'empty sll'
        res=''
        while itr:
            res+=str(itr.val)+'-->' if itr.next else str(itr.val)
            itr=itr.next
        return res
    def sort(self):
        itr=self.head
        while itr:
            org=itr
            temp=itr.next
            while temp:
                if temp.val<org.val:
                    org=temp
                temp=temp.next
            x=itr.val
            itr.val=org.val
            org.val=x
            itr=itr.next
obj=LinkedList()
```

In [34]:
```
arr=[4,2,-1,0,3,8,5,6]
obj.head=ListNode(arr[0])
temp=obj.head
for i in range(1,len(arr)):
    temp.next=ListNode(arr[i])
    temp=temp.next
```

In [35]:
```
obj.printList()
```

Out[35]:
```
'4-->2-->-1-->0-->3-->8-->5-->6'
```

In [36]:
```
obj.head.next.next.val
```

Out[36]:
```
-1
```

In [37]:
```
obj.sort()
```

In [38]:
```
obj.printList()
```

Out[38]:
```
'-1-->0-->2-->3-->4-->5-->6-->8'
```

In [ ]:

In [ ]:

In [ ]:

In [ ]: